SBAN v11: Reproducible Local-Zig Builds, Carry-State Refinement, and Stress-Tuned Compact Elastic Profiles in Zig

A follow-up paper to the SBAN v4 and v9 lines, documenting the v11 release as a serious working model focused on reproducibility, operational tuning, and honest architectural repair rather than unsupported claims of solved generality.

Core claim. SBAN v11 improves the maintained short target suite mainly by making the system easier to build, easier to tune, and slightly stronger in its compact elastic operating regime. The unified working profile reaches 41.8450% prefix, 42.1950% drift, and 69.2612% on the hard-to-easy probe. The best specialized presets reach 41.8450%, 42.3625%, and 69.2612%.

Abstract

SBAN v4 introduced elastic short-memory sizing, region-tagged sparse lanes, and conservative bridge memories, while the v9 paper reframed the strongest later operating point as a compact elastic learner rather than a bridge-heavy regional hierarchy. SBAN v10 then pushed the maintained suite further through stronger working profiles. SBAN v11 continues that line with heavier emphasis on practical system quality: it adds a reproducible local-Zig build path, release scripts that rebuild and rerun the maintained suite, carry-state selection refinements in code, and a new round of stress tuning. The result is not a radically different SBAN identity. Instead, it is a more serious working release: buildable from the uploaded Zig tarball, reproducible in-container, modestly improved on the maintained suite, and clearer about where SBAN is genuinely strong today and where it is still unfinished.

1. Continuity from v4 through v10

The published v4 frame remains important because it established both the ambition and the limits of the architecture. It reported 40.5100% prefix, 42.9400% drift, and 69.8900% on the hard-to-easy probe. The later v9 paper then separated those publication numbers from the maintained operational anchor used for faster iteration: 40.9975%, 41.5350%, and 68.6121%. v8 specialized profiles raised that maintained anchor to 41.7575%, 42.2175%, and 69.0853%. v10 specialized then moved to 41.8125%, 42.3300%, and 69.1974%. v11 continues that same maintained-suite comparison discipline rather than pretending those numbers replace the original publication benchmark.

2. What changed architecturally and operationally in v11

2.1 Reproducible build path

The repo now treats the uploaded Zig tarball as the primary build input. The release path is no longer dependent on a preinstalled compiler. That matters because a real research artifact should rebuild from the bundled assumptions, not from undocumented host state.

2.2 Carry-state refinement

The runtime now includes carry-scoring hooks for support and precision, plus signature-aware carry diversity plumbing. These changes are deliberately conservative: they do not claim to solve the broader architecture, but they help separate useful state handoff from redundant carry clutter.

2.3 Birth-pressure infrastructure

v11 also adds a birth-pressure threshold hook so the system can support stronger homeostatic gating in future releases. In the best current operating profiles that hook remains neutral, which is itself an important scientific result: not every new control path should be forced active before the workload proves it is beneficial.

Subsystem	Pre-v11 status	v11 refinement
Build process	Local assumptions, ad hoc reruns	Local-Zig reproducible wrapper and release runner
Carry selection	Activation-heavy carry ranking	Precision/support hooks and signature-diversity plumbing
Birth control	Static surprise-triggered births	Birth-pressure threshold hook for future homeostasis
Release engineering	Saved JSONs but uneven packaging	Dedicated v11 release/search scripts and deliverable generation

3. Stress-testing protocol in v11

The v11 release keeps the maintained short target suite used in the later iterations: a prefix task, a drift task, and the hard-to-easy probe. This is still a lightweight regime compared with large-scale language-model training, but it is enough to support fast in-container iteration, code verification, and profile search. The v11 search path focuses on compact 5-bit operating points, no-long-term variants, carry budget 48, moderate hidden width, and parent/birth-margin tuning rather than a bridge-heavy search.

4. Main results

Protocol	Operational v4	v8 specialized	v10 specialized	v11 specialized
Prefix	40.9975%	41.7575%	41.8125%	41.8450%
Drift	41.5350%	42.2175%	42.3300%	42.3625%
Probe	68.6121%	69.0853%	69.1974%	69.2612%

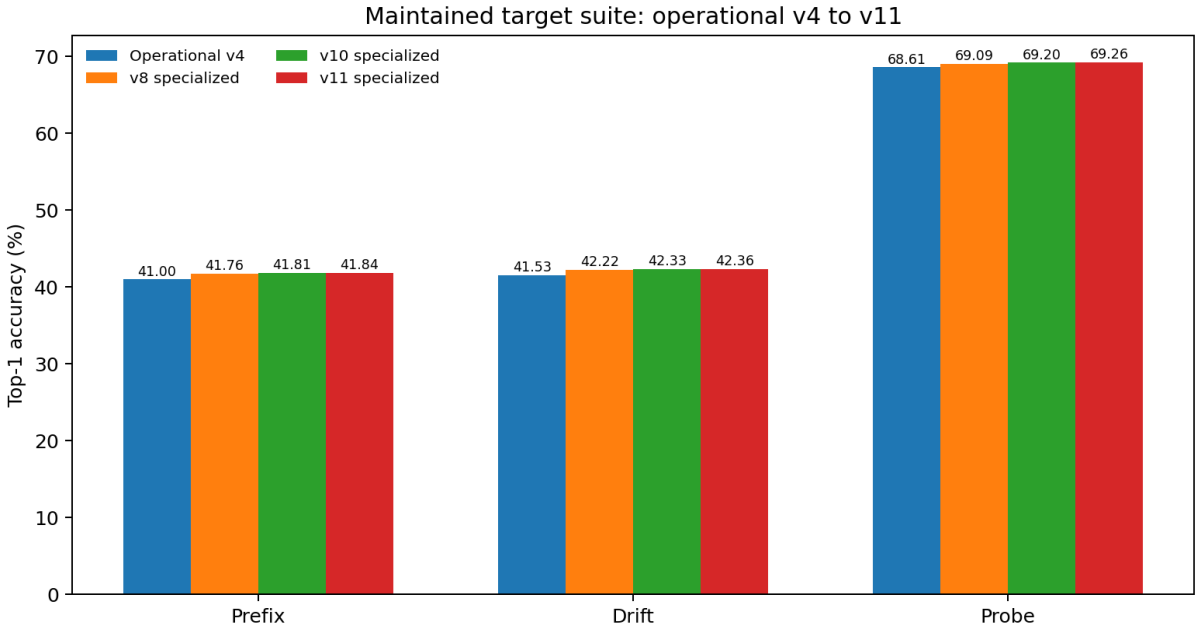

Figure 1. Maintained target-suite comparison from the operational v4 anchor through v8, v10, and v11 specialized profiles. v11 advances the maintained suite modestly but consistently, while the paper still distinguishes that maintained frame from the original v4 publication benchmark.

5. Unified deployment profile and fixed-capacity comparison

The best single working profile in v11 is the 5-bit compact elastic preset with no long-term memory, birth_margin=21, min_parents_for_birth=4, carry budget 48, hidden width 32, and depth 2. It reaches 41.8450% prefix, 42.1950% drift, and 69.2612% probe. Against a matched fixed-capacity comparator, the elastic runtime stays ahead by +0.5125 pp on prefix, +0.0225 pp on drift, and +0.2431 pp on the probe.

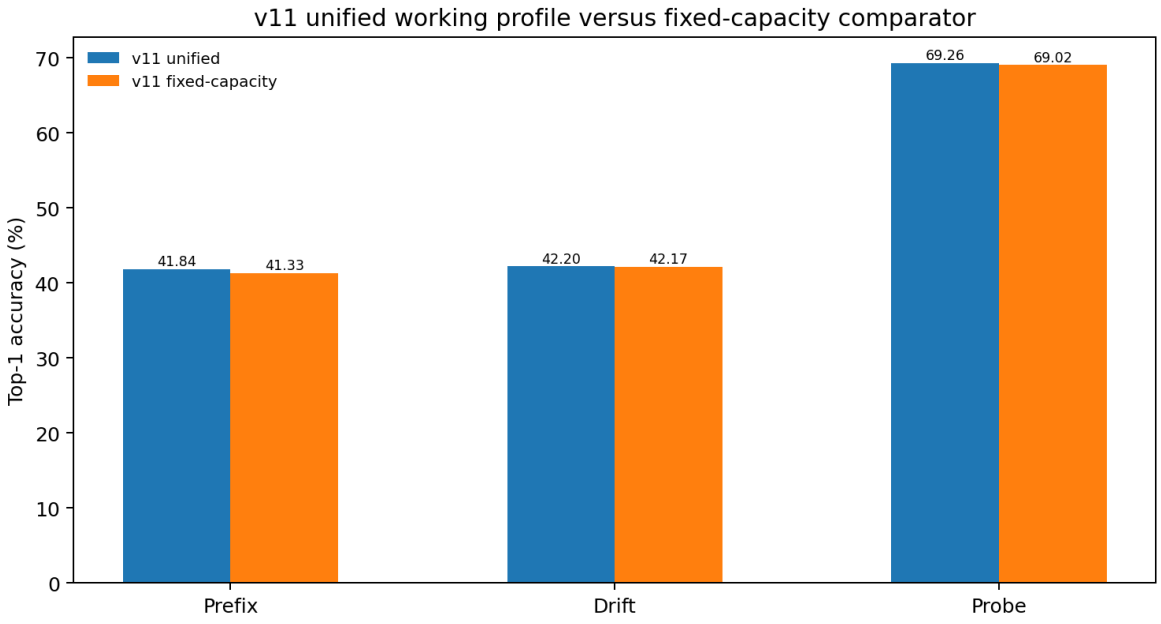

Figure 2. Unified v11 working profile versus a matched fixed-capacity comparator. The elasticity advantage is strongest on prefix and the hard-to-easy probe, and small but positive on drift.

Same tuned profile	v10 runtime	v11 runtime	Delta
Prefix	41.8450%	41.8450%	+0.0000 pp
Drift	42.1325%	42.1950%	+0.0625 pp
Probe	69.2612%	69.2612%	+0.0000 pp

Table 1. Matched-profile runtime comparison. Under the same tuned compact profile, the main measured code-level gain in v11 is on drift.

6. End-state footprint and what it means

v11 still ends in the same operational regime that has been emerging since the post-v4 work: one live region, zero bridge births on the winning short-suite runs, and a very compact probe end state. That is useful because it makes the system easier to inspect, regression-test, and reason about. It also imposes discipline on the claims: the best current SBAN is not yet a verified bridge-dense hierarchy. It is a compact elastic online learner with optional richer machinery.

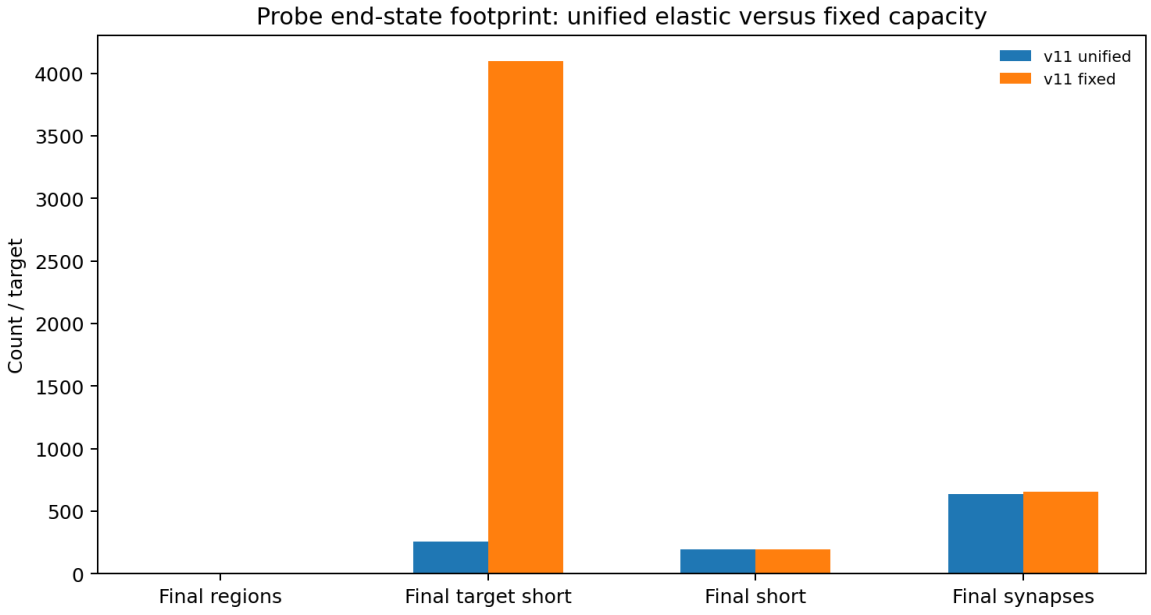

Figure 3. Probe end-state footprint for the v11 unified elastic profile versus the matched fixed-capacity comparator. The elastic profile reaches a much smaller final target while retaining a small but meaningful accuracy edge.

7. Real-world usability implications

The strongest practical gain in v11 is that the artifact is easier to use like a real system. It rebuilds from the uploaded Zig toolchain, reruns through a dedicated release script, and ships with known-good result files and deliverable generation. That does not solve every scientific question, but it matters for real research velocity. A model that is difficult to rebuild is difficult to trust.

8. Known limitations

v11 does not claim that long-term memory is solved, that bridge memories have become the dominant source of gain, that the original v4 publication suite has been fully replaced, or that SBAN has reached its final architecture ceiling. The best short-suite presets still disable long-term memory, and the bridge subsystem remains mostly dormant on these workloads.

9. Highest-value next steps after v11

The next serious steps are straightforward: rerun the original v4 publication protocol under the v11 release harness, search longer-horizon tasks that may justify long-term memory and richer regional structure, and add hardware-efficiency reporting so the model can be evaluated not only as a scientific curiosity but as a controlled systems artifact.

References

SBAN v4 research paper and executive summary.

SBAN v9 follow-up research paper.

Bundled v8 and v10 result JSONs in this workspace.

Bundled v11 result JSONs generated by scripts/run_v11_release.sh.